



Penyusun:

Satria Nazar Alief 2403015060

Dosen Pengampu : Ir. Arry Avorizano, S.Kom., M.Kom.

**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNOLOGI INDUSTRI DAN
INFORMATIKA UNIVERSITAS
MUHAMMADIYAH PROF.DR HAMKA**

Pertemuan 7 – Perancangan Arsitektur Perangkat Lunak

I. TUJUAN INSTRUKSIONAL

Setelah mengikuti sesi pelatihan ini, peserta diharapkan mampu:

1. Menjelaskan secara komprehensif struktur arsitektur perangkat lunak yang akan dikembangkan.
2. Memetakan hubungan dan interaksi antar komponen sistem dengan jelas.
3. Membimbing proses implementasi kode agar sesuai dengan cetak biru (*blueprint*) arsitektur guna memudahkan pengembangan dan pemeliharaan (*maintenance*) sistem.

II. MATERI / TEORI SINGKAT (Bahan Ajar Trainer)

Arsitektur perangkat lunak merupakan rancangan struktur sistem secara keseluruhan yang menggambarkan bagaimana komponen-komponen dalam sistem saling berinteraksi.

Sebagai *developer* maupun penguji, pemahaman terhadap arsitektur sistem sangat krusial untuk membagi tugas pengerjaan setiap bagian sistem agar proses pengembangan menjadi lebih terstruktur, mudah diskalakan (*scalable*), dan adaptif terhadap perbaikan kesalahan (*debugging*).

Beberapa model arsitektur standar industri yang wajib dikuasai peserta:

- **Layered Architecture:** Pembagian sistem menjadi beberapa lapisan logis terpisah, seperti *presentation layer*, *business/application layer*, dan *database/data access layer*.
- **MVC (Model-View-Controller):** Pola arsitektur yang memisahkan antara manajemen data (Model), antarmuka pengguna (View), dan logika bisnis (Controller).
- **Client-Server:** Model arsitektur di mana *client* berperan mengirimkan permintaan (*request*) dan *server* bertanggung jawab memproses data serta mengembalikan tanggapan (*response*).

III. ALAT DAN BAHAN PELATIHAN (Syarat Kelas)

Sebelum memulai sesi praktikum, Trainer wajib memastikan peserta telah menyiapkan:

- Laptop/Komputer dengan koneksi internet aktif.
- Akses ke Modul RPL (Rekayasa Perangkat Lunak).
- Perangkat lunak pemodelan / UML Tool (Draw.io, StarUML, atau Astah).
- Web Browser (Google Chrome / Mozilla Firefox).
- Akun GitHub aktif untuk manajemen *repository*.

IV. STUDI KASUS & CONTOH PERMASALAHAN (Panduan Analisis Kelas)

Problem Statement untuk Diskusi Kelas: Banyak tim pengembang pemula mengalami kegagalan/keterlambatan rilis karena tidak merancang struktur sistem sejak awal. Dampaknya, kode program menjadi berantakan (*spaghetti code*), sulit diperbaiki jika ada *bug*, dan sistem mengalami *crash* saat ingin ditambahkan fitur baru.

Objek Proyek Simulasi:

- **Tema Proyek:** Website Pre-Order Toko Kaizen
- **Nama Sistem:** Sistem Website Pre-Order Toko Kaizen
- **Tujuan Sistem:** Memfasilitasi pelanggan untuk memesan produk secara *pre-order* via online, serta menyediakan dasbor bagi pemilik toko untuk mengelola data pesanan secara terstruktur dan efisien.
- **Target Pengguna:** Pelanggan (End-User) dan Admin Toko.
- **Tautan Kerja Kelompok (GitHub):**

[<https://github.com/palentpemberani/KAIZEN-CAKERY-4A.git>] (<https://github.com/palentpemberani/KAIZEN-CAKERY-4A.git>)

V. MATERI INTI: PERANCANGAN ARSITEKTUR SISTEM

(Trainer menjelaskan blueprint sistem menggunakan pendekatan Client-Server berbasis Layered Architecture)

1. Client Layer (Presentation Layer)

Lapisan terdepan yang berinteraksi langsung dengan pengguna melalui *browser*.

- **Fitur yang Divalidasi:** Halaman Katalog Produk, Sistem Autentikasi (Login/Registrasi), Formulir Pemesanan, dan Halaman Status Pesanan.
- **Aktor:** Pelanggan & Admin.

2. Application Layer (Business Logic Layer)

Lapisan inti tempat seluruh aturan bisnis dan pemrosesan data dijalankan.

- **Modul Utama:** *Auth Module*, *Order Module*, *Product Module*, dan *Status Module*.
- **Tugas Sistem:** Memvalidasi input pengguna, mengolah logika pesanan, dan menjembatani komunikasi ke penyedia data.

3. Database Layer (Data Access Layer)

Lapisan penyimpanan data akhir. Pada proyek ini menggunakan **Google Sheets API** sebagai *data store* utama.

VI. DIAGRAM ARSITEKTUR & ALUR PROSES (Instruksi Trainer)

(Tampilkan diagram arsitektur sistem pada slide/layar proyektor kepada peserta)

Penjelasan Alur Kerja Sistem (Delivery Point untuk Trainer):

1. **Akses Awal:** Trainer menginstruksikan bagaimana pengguna mengakses halaman website via browser.
2. **Autentikasi & Input:** Pengguna melakukan login, memilih produk, dan mengisi data pada form pesanan.
3. **Transport Data:** Browser mengirimkan data tersebut dalam bentuk **HTTP Request** menuju *Application Layer*.
4. **Processing:** *Application Layer* melakukan validasi data dan menjalankan fungsi logika bisnis.
5. **Storage:** Data yang telah valid dikirim ke **Google Sheets** menggunakan *Google Sheets API*.
6. **Admin Monitoring:** Admin/Pemilik toko dapat memantau data pesanan langsung melalui *spreadsheet* dan memperbarui status pesanan di sana.
7. **Real-time Feedback:** Sistem membaca pembaruan dari *spreadsheet* tersebut, mengirimkan **HTTP Response**, dan menampilkannya kembali ke layar browser pelanggan secara *real-time*.

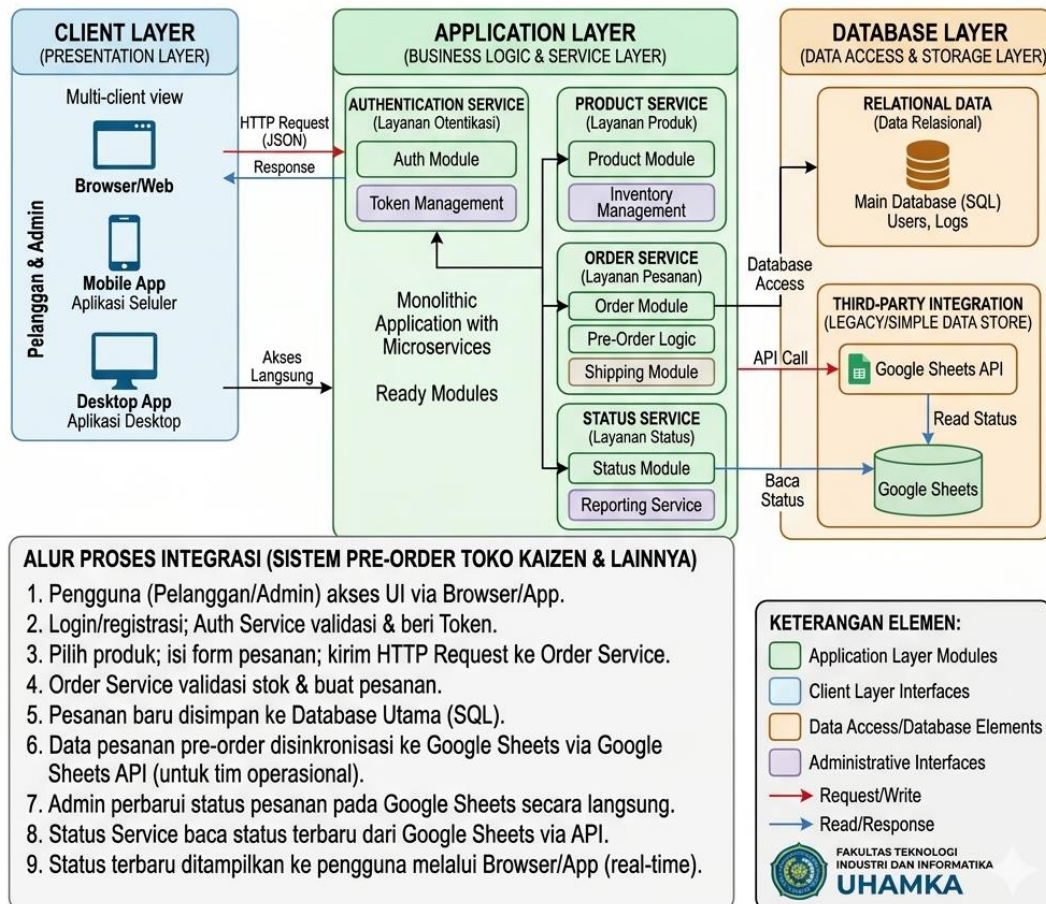
Berikut adalah komponen visual yang ada di dalam gambar tersebut untuk memastikan Anda tidak salah memasukkan file:

- **Sisi Kiri (Client Layer):** Kotak bertuliskan *Browser/UI (Pelanggan & Admin)*.
- **Sisi Tengah (Application Layer):** Kotak-kotak modul program seperti *Auth Module, Order Module, Product Module*, dan *Status Module*.
- **Sisi Kanan (Database Layer):** Kotak bertuliskan *Google Sheets API* dan *Google Sheets (Penyimpanan Data)*.
- **Alur Panah:** Panah bolak-balik yang menandakan aliran *HTTP Request/Response* serta *API Call*.



PEDOMAN PELATIHAN: DIAGRAM ARSITEKTUR SISTEM PERANGKAT LUNAK (UHAMKA)

PELATIHAN: PRINSIP DAN POLA ARSITEKTUR PERANGKAT LUNAK (PELANGGAN SEBAGAI TRAINER)



VII. TUGAS PRAKTIKUM & POIN PENILAIAN

Sebagai Trainer, berikan instruksi tugas berikut kepada peserta di akhir sesi:

1. **Tugas Mandiri:** Setiap kelompok wajib membuat diagram arsitektur sistem untuk proyek mereka masing-masing menggunakan Draw.io/StarUML.
2. **Kriteria Penilaian:**
 - o Kejelasan pemisahan antar *layer* (Client, Application, Database) (Bobot: 30%)
 - o Ketepatan arah panah *HTTP Request/Response* dan *API Call* (Bobot: 40%)
 - o Kesesuaian arsitektur dengan fungsionalitas kode di repositori GitHub mereka (Bobot: 30%)